

基于Easyx库的C++吃豆人小游戏

计科2404

24 杨

2025年5月8日

项目地址: <https://github.com/jnu-dreamer/Pacman-EasyX>

讲解视频地址: <https://www.bilibili.com/video/BV1oT7tzAEM1/>

目录

content

1. EasyX库简介
2. 游戏逻辑简介
3. 双缓冲机制
4. 寻路方式优化

① EasyX库简介

EasyX Graphics Library 是针对 Visual C++ 的免费绘图库，支持 VC6.0 ~ VS2022，简单易用，学习成本极低，应用领域广泛。

EasyX提供了绘图、文字处理、消息处理、图片处理等诸多功能的函数，使用时需引用 `graphics.h` 或 `easyx.h` 头文件。具体用法参见官方文档：
<https://docs.easyx.cn/zh-cn/intro>

② 游戏逻辑说明

游戏主体为简单的吃豆人玩法，包含玩家（player）、鬼怪（ghost）、地图（map）等元素。

在游戏中，玩家需要吃掉所有豆子来获得胜利，同时也要躲避鬼怪的追捕。游戏设有三个不同难度，不同难度的鬼怪速度，玩家生命有所不同。

游戏中同时设有超级豆，在吃掉超级豆后的 5s 内，玩家进入无敌状态，同时鬼怪会“害怕”而远离玩家。

程序中部分使用了 OOP 的编程思想，将 ghost、player 封装成类，并皆由抽象基类 mover 派生而来。

② 游戏逻辑说明

mover基类，其中包含纯虚函数update()，
由派生类重写

```
class mover
{
protected:
    int x = 14, y = 14;
    pair<int, int> dir = { 0,0 };
    int speed = 0;
    IMAGE* img = nullptr;
public:
    mover(){} // 默认构造函数
    mover(int x, int y, int speed = 0, IMAGE* image = nullptr) :
        x(x), y(y), speed(speed), img(image) {} // 重载构造函数

    virtual void update() = 0;
    void draw(int idx) { ... }
    void setSpeed(int s) { this->speed = s; }
    void setX(int newX) { this->x = newX; }
    void setY(int newY) { this->y = newY; }
    void setImg(IMAGE* newimg) { this->img = newimg; }
    int getX() const { return x; }
    int getY() const { return y; }
};
```

② 游戏逻辑说明

派生类Player

```
class Player : public mover
{
private:
    // ... 生命、得分等私有成员
public:
    Player() {}
    Player(int x, int y, int speed = 0, IMAGE* img = nullptr) : mover(x, y, speed, img) {}

    void reset() { ... }
    void update() override { ... } // 虚函数实现, 更新玩家状态
    bool collision(int ghostX, int ghostY) { ... } // 碰撞检测
    void die() { ... } // 判定死亡

    // ... 私有成员查询与修改接口
};
```

② 游戏逻辑说明

派生类Ghost

```
class Ghost : public mover
{
public:
    Ghost(int x, int y, int speed = 1, IMAGE* image = nullptr) :
        mover(x, y, speed, image) {}

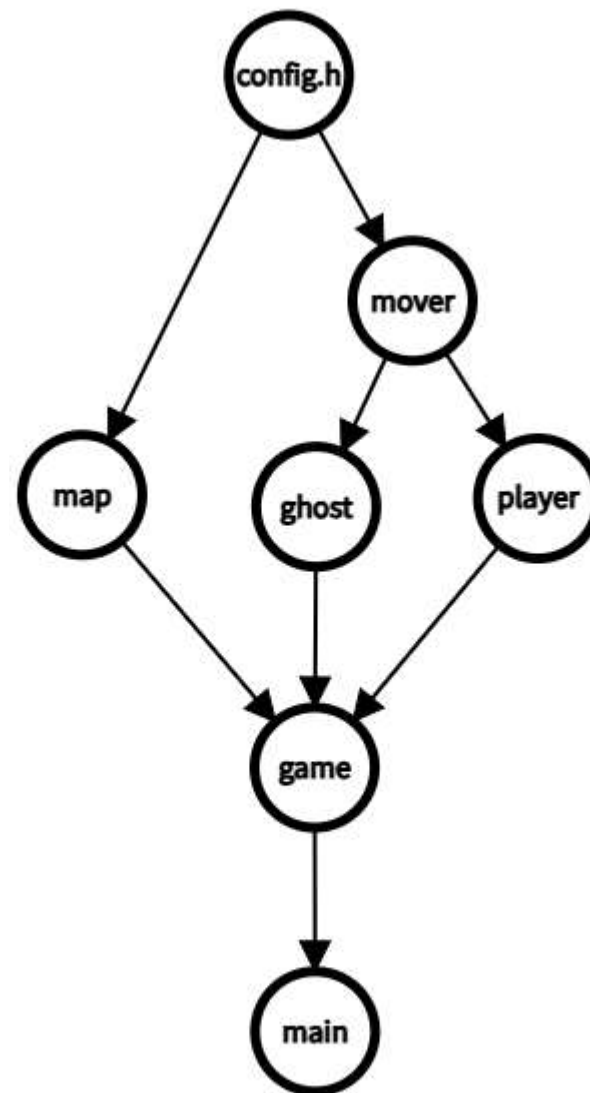
    void reset(int setx, int sety) { ... } // 重设状态
    pair<int, int> bfs(int playerX, int playerY) { ... } // 寻找最短路, 返回下一步方向
    void update() override { ... } // 随机更新方向
    void update(int playerX, int playerY) { ... } // bfs更新方向
    void circle() { ... } // 巡航移动
};
```


② 游戏逻辑说明

游戏主要逻辑如下：

```
// ... 导入图片、变量初始化等
while (true)
{
    startTime = clock();

    Input(); // 处理输入
    updateGame(); // 更新游戏状态
    renderGame(); // 渲染游戏画面
    controlFPS(startTime); // 控制帧率
    frame++;
}
```



③ 双缓冲机制

如果使用传统的绘制流程，会经历如下步骤：

清空屏幕->绘制地图->绘制角色->绘制鬼怪，
且都在一个窗口上执行。

在这过程中很容易看到绘制的中间状态，即导致画面闪烁。

而双缓冲则在后台另外开辟了一个缓冲区，将所有内容先绘制到这个缓冲区内，再将缓冲区整体搬到展示的窗口上。

EasyX提供了 `BeginBatchDraw()` 和 `EndBatchDraw()` 这两个函数，便捷地实现了双缓冲。

③ 双缓冲机制



双缓冲绘图的实现

```
void renderGame()
{
    BeginBatchDraw();
    cleardevice();
    if (gameState == READY)
    {
        StartScreen(); // 开始菜单
        EndBatchDraw();
        return;
    }
    if (gameState == LOSE)
    {
        GameOver(); // 游戏结束菜单
        EndBatchDraw();
        return;
    }
    if (gameState == WIN)
    {
        GameWin(); // 胜利菜单
        EndBatchDraw();
        return;
    }
    drawMap();
    // ... 其余界面绘制
    EndBatchDraw();
}
```

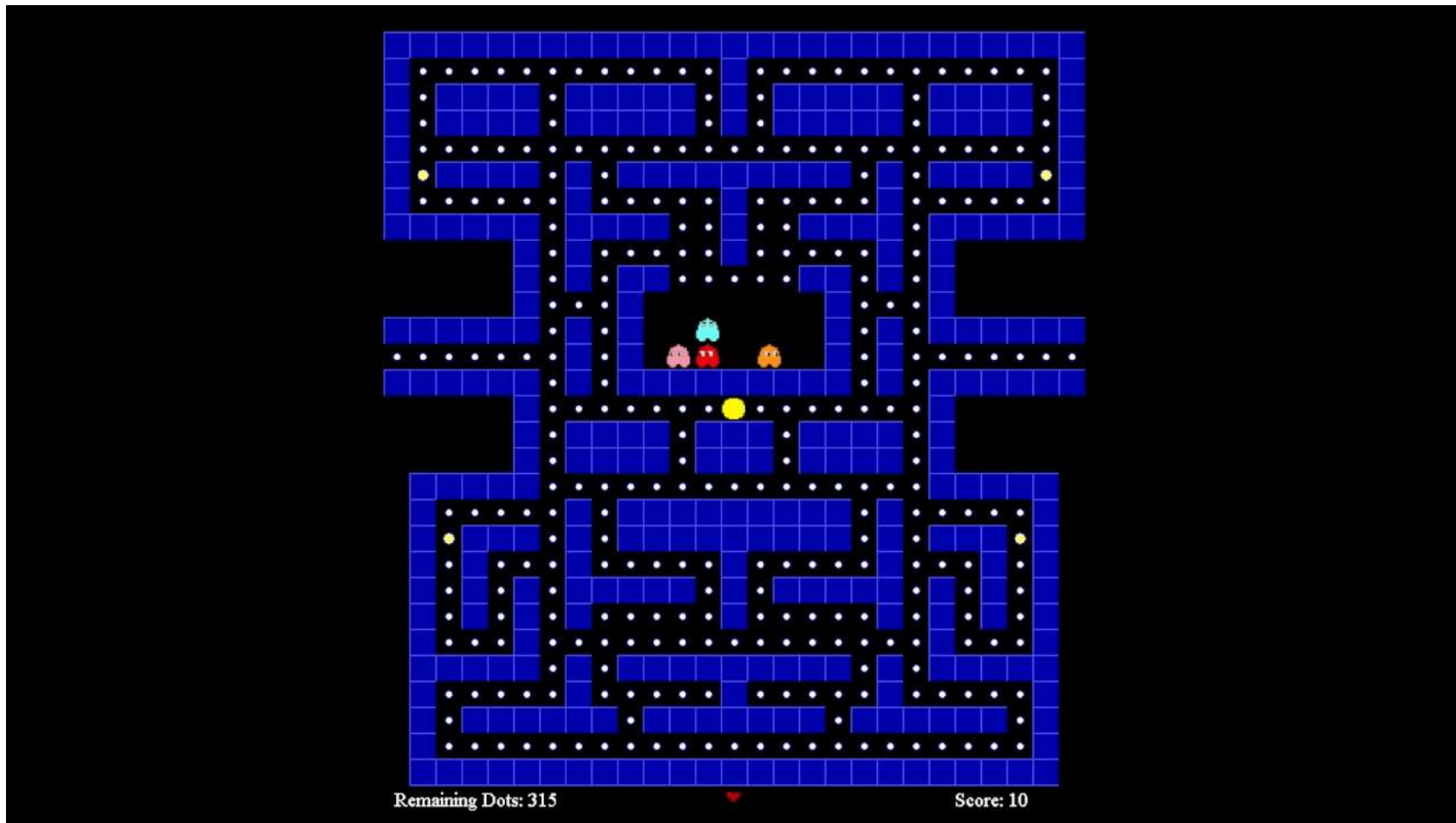
④ 寻路方式优化

首先是最朴素的想法，随机寻路，但这样就像无头苍蝇，几乎不会碰到player。

```
void update() override
{
    // 在格子中间10%概率随机改变方向
    if (rand() % 10 == 0 && x % block_size == 0 && y % block_size == 0)
    {
        // 获取可行方向
        vector<pair<int, int>> validDirs;
        for (const auto& d : DIRECTIONS) {
            int newX = x + d.first * speed;
            int newY = y + d.second * speed;
            if (canMoveTo(newX, newY)) {
                validDirs.push_back(d);
            }
        }
        // 如果有可行方向，随机选择一个
        if (!validDirs.empty())
            dir = validDirs[rand() % validDirs.size()];
        // ... 剩余移动逻辑
    }
}
```

④ 寻路方式优化

随机寻路



④ 寻路方式优化

```
pair<int, int> bfs(int playerX, int playerY)
{
    // ... 转化坐标与比较

    bool vis[ROWS][COLS] = {false}; // 是否访问过
    pair<int, int> pre[ROWS][COLS]; // 记录路径
    for (int i = 0; i < ROWS; i++) for (int j = 0; j < COLS; j++) pre[i][j] = { -1, -1 };

    queue<pair<int, int>> q; q.push({ sr, sc }); vis[sr][sc] = true;
    while (!q.empty()) {
        auto [r, c] = q.front(); q.pop(); // 结构化绑定
        if (r == tr && c == tc) break;
        for (auto [dx, dy] : DIRECTIONS) {
            int nr = r + dy, nc = c + dx;
            if (nr >= 0 && nr < ROWS && nc >= 0 && nc < COLS && !vis[nr][nc] && mp[nr][nc] != 1)
                vis[nr][nc] = true, pre[nr][nc] = { r, c }, q.push({ nr, nc });
        }
    }
    if (pre[tr][tc] == make_pair(-1, -1)) return { 0, 0 };
    int r = tr, c = tc; while (pre[r][c] != make_pair(sr, sc)) r = pre[r][c].first, c = pre[r][c].second;
    return { c - sc, r - sr };
}
```

为了解决这个问题，我们引入了BFS（广度优先搜索）算法，用于搜索鬼怪到玩家的最短路及下一步的方向。

但是这样又会带来新的问题，四只鬼怪用相同的逻辑导致路线也相同，只有一条线追踪，趣味性低。



因此，我们通过随机和最短路的结合，使不同鬼怪的移动速度不一样，并且既能分散又能实现追逐玩家的效果。

```
if (!player.isinvincible())
{
    if (i == 0 ) ghosts[i].circle();
    else if (i == 1 && frame % 100 <= 30) ghosts[i].update();
    else ghosts[i].update(player.getX(), player.getY());
}
else ghosts[i].antibfs(player.getX(), player.getY()); // 无敌状态下鬼怪逃跑
```


随机和最短路的结合



```
void circle()
```

```
{
```

```
    // 判断格子中心
```

```
    static const vector<pair<int, int>> dirs = { {0,-1}, {1,0}, {0,1}, {-1,0} };
```

```
    // 当前方向在dirs中的下标
```

```
    int curIdx = 0;
```

```
    for (int i = 0; i < 4; ++i) if (dir == dirs[i]) { curIdx = i; break;}
```

```
    // 左转、直行、右转、掉头的优先级
```

```
    int tryOrder[4] = { (curIdx + 3) % 4, curIdx, (curIdx + 1) % 4, (curIdx + 2) % 4 };
```

```
    for (int i = 0; i < 4; i++)
```

```
    {
```

```
        auto d = dirs[tryOrder[i]];
```

```
        int newX = x + d.first * block_size;
```

```
        int newY = y + d.second * block_size;
```

```
        if (canMoveTo(newX, newY)) {dir = d; break;}
```

```
    }
```

```
    // 移动
```

```
    x += dir.first * speed;
```

```
    y += dir.second * speed;
```

```
}
```

此外，我们还参考原作，实现了鬼怪绕着障碍物
巡航的更新逻辑和玩家无敌时逃跑的更新逻辑
(反向的BFS)

```

void antibfs(int playerX, int playerY)
{
    // 判断中心和转换坐标

    // 1. BFS从玩家出发, 计算每个格子到玩家的最短距离
    int dist[ROWS][COLS];
    for (int i = 0; i < ROWS; ++i)
        for (int j = 0; j < COLS; ++j)
            dist[i][j] = -1;

    queue<pair<int, int>> q;
    q.push({ playerR, playerC });
    dist[playerR][playerC] = 0;

    static const vector<pair<int, int>> dirs = { ... }

    while (!q.empty())
    {
        auto [r, c] = q.front(); q.pop();
        for (auto& d : dirs) { ... }
    }
}

```

```

// 2. 选择距离玩家最远的方向
int maxDist = dist[curRow][curCol];
pair<int, int> bestDir = dir;
vector<pair<int, int>> validDirs;
for (const auto& d : dirs)
{
    int newRow = curRow + d.second;
    int newCol = curCol + d.first;
    if (newRow >= 0 && newRow < ROWS && newCol >= 0
        && newCol < COLS && mp[newRow][newCol] != 1)
    {
        validDirs.push_back(d);
        if (dist[newRow][newCol] != -1
            && dist[newRow][newCol] > maxDist) { ... }
    }
}

// 如果没有更远的方向, 随机选一个可行方向, 避免卡住
if (bestDir == dir && !validDirs.empty()) { ... }

dir = bestDir;
int newX = x + dir.first * speed;
int newY = y + dir.second * speed;
if (canMoveTo(newX, newY)) { ... }

```